

GRAPE for Transmon Qubits

grape_scq

L. Van Damme

September 7, 2026

Contents

Overview	2
Requirements	2
Object model	3
Example 1: Leakage-Free NOT Gate	4
Example 2: Two-Photon Transition	6
Example 3: Analytically Shaped Hadamard Gate	7
Example 4: Robust Fourier Pulse	9
Example 5: Selective Pulse	11
Example 6: Time-Dependent Detuning Robustness	12

Overview

This Python package provides tools to optimize control pulses for transmon qubits with GRAPE (GRAdient Ascent Pulse Engineering). The qubit dynamics are described by the Hamiltonian in the rotating-wave approximation (with $\hbar = 1$):

$$H/(2\pi) = \frac{\nu_{\text{Ref}}}{2} \left(u(t) a^\dagger + u^*(t) a \right) + (\nu_q - \nu_d) a^\dagger a + \frac{\alpha}{2} a^{\dagger 2} a^2, \quad (1)$$

where:

- ν_{Ref} is a reference frequency (Hz) that sets the overall drive scale.
- $u(t) = u_x(t) + i u_y(t)$ is the normalized control pulse in complex form, with $u_x(t)$ (in-phase) and $u_y(t)$ (quadrature) real and dimensionless.
- ν_q is the qubit frequency (the $|0\rangle \leftrightarrow |1\rangle$ transition) in Hz.
- ν_d is the carrier (drive) frequency in Hz.
- α is the transmon anharmonicity in Hz.
- a and $a^\dagger$ are the annihilation and creation operators, truncated to the number of levels considered.

Any of ν_q , α , or the drive amplitude scale factor may be a scalar, an array over an *ensemble* of parameter values (robust optimization against uncertain parameters), or an array over both an ensemble and time (e.g. a drifting or oscillating detuning). Control pulses can either be optimized directly at every timestep or restricted to a linear combination of analytical basis functions, to produce experimentally feasible shapes and to reduce the number of free parameters. Pulses can be constrained to respect hardware amplitude limits. The optimizer can target either a unitary gate on a computational subspace or a state-to-state transfer in the full Hilbert space.

Limitations. The code supports only single or uncoupled transmons, does not implement multi-qubit gates, does not allow for multiple control channels with different drive frequencies, neglects relaxation processes, and is valid only within the rotating-wave approximation.

Engine. The cost function and its exact gradient (via the adjoint-state method, one eigendecomposition per timestep) are compiled with Numba, so the optimizer scales well with the number of timesteps, ensemble members, and analytic-basis coefficients.

Requirements

This package requires numpy, numba, scipy (in particular `scipy.optimize`), and matplotlib.

Object model

A problem is assembled from a small number of explicit, composable objects:

- `Transmon` — the physical system: `n_levels`, `frequency`, `anharmonicity`, `amplitude_scale`, `carrier_frequency`.
- `GateTarget(matrix, subspace)` / `StateTarget(initial, target)` — the optimization target.
- `OptimizationProblem(system, target, n_steps, dt, ...)` — composes the system, the target, the time grid and, optionally, an analytic pulse basis (`ux_basis` / `uy_basis`).
- `GrapeOptimizer(max_iter=..., ...)` — `optimizer.optimize(problem, initial_pulse)` returns a `Result`.
- `initial_guess` — generators (`cos_drag`, `random_fourier`, `random_symmetric_antisymmetric`, `constant`, `zero`) producing an initial `Pulse`.
- `Result` — the optimized pulse and convenience methods: `.plot_pulse()`, `.population(...)`, `.fidelity_map()`, `.summary()`.

Example 1: Leakage-Free NOT Gate

`examples/01_leakage_free_gate.ipynb`

General problem. We consider a three-level system driven on resonance. The system Hamiltonian under the rotating-wave approximation is given by

$$H(t) = \frac{2\pi\nu_{\text{Ref}}}{2} (u(t) a^\dagger + u^*(t) a) + \frac{2\pi\alpha}{2} a^\dagger a^2,$$

where $\alpha = -100$ MHz and ν_{Ref} is set to 10 MHz. To account for three energy levels, the operators a and $a^\dagger$ are truncated as

$$a = |0\rangle\langle 1| + \sqrt{2}|1\rangle\langle 2|, \quad a^\dagger = |1\rangle\langle 0| + \sqrt{2}|2\rangle\langle 1|.$$

The goal is to optimize the pulse shape $u(t)$ to implement the gate

$$U_T = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

on the computational subspace defined by $|0\rangle$ and $|1\rangle$. The time is discretized into 120 steps of 0.1 ns, corresponding to a 12 ns-long control pulse.

From a control perspective, this problem is underconstrained, meaning that infinitely many pulse shapes can reach the target to numerical precision. To select among these degenerate solutions, we minimize the pulse energy

$$(2\pi\nu_R)^2 \int_0^T u_x^2(t) + u_y^2(t) dt,$$

which restricts the search to solutions that achieve the target while using minimal control amplitude.

Problem setup.

```
system = Transmon(  
    n_levels=3,                # Number of energy levels  
    anharmonicity=-100e6,      # Anharmonicity in Hz  
)  
target = GateTarget(  
    matrix=[[0, 1], [1, 0]],   # Target gate (NOT gate)  
    subspace=[0, 1],          # Computational states indices  
)  
problem = OptimizationProblem(  
    system, target,  
    n_steps=120,               # Number of time steps  
    dt=0.1e-9,                 # Timestep in s.  
    reference_amplitude=10e6,  # Reference pulse amplitude  
)
```

Note that the carrier and qubit frequencies do not need to be specified for this simple problem, as the drive is assumed to be on-resonance with the qubit transition by default.

Initial pulse guess. We use a DRAG-like pulse of the form

$$u_{x,i}(t) = A \sin\left(\frac{\pi t}{T}\right), \quad u_{y,i}(t) = B \cos\left(\frac{\pi t}{T}\right),$$

where the parameters A and B are internally fit to minimize leakage under the problem's own system and duration, generated with:

```
pulse0 = initial_guess.cos_drag(problem, theta=np.pi)
```

Here, `theta=np.pi` specifies that the DRAG guess should implement a π -rotation in the computational basis.

Optimization. We search for a minimum-energy solution:

```
optimizer = GrapeOptimizer(
    max_iter=500,          # Maximum number of iterations
    minimize_energy=True,  # Find minimum-energy solution
)
result = optimizer.optimize(problem, pulse0)
```

which returns a `Result` holding the optimal pulse and its fidelity.

Results. The pulse components in Hz are obtained and plotted with:

```
result.plot_pulse(compare_to_initial=True)
```

The resulting pulse is shown in Fig. 1, together with the leakage population $|\langle 2|\psi(t)\rangle|^2$ obtained from `result.population(0, 2)` and `result.population(1, 2)`. The optimized pulse reaches a gate fidelity of 100.000% (to the precision shown), essentially suppressing leakage to $|2\rangle$, starting from a 98.005%-fidelity DRAG guess. The complete script is provided in `examples/01_leakage_free_gate.ipynb`.

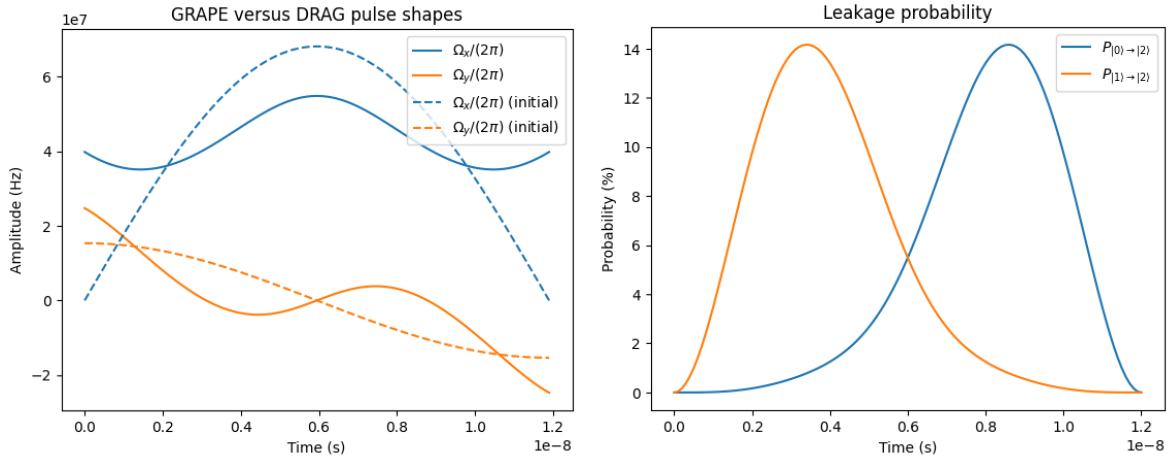


Figure 1: Left panel: Pulse shapes in Hz before (dashed) and after (solid) optimization. Right panel: Leakage to state $|2\rangle$ as a function of time.

Example 2: Two-Photon Transition

examples/02_two_photon_transition.ipynb

General problem. We consider a transmon with anharmonicity $\alpha = -200$ MHz and $|0\rangle \leftrightarrow |1\rangle$ frequency $\nu_q = 5$ GHz, driven at the two-photon resonance

$$\nu_d = \nu_q + \alpha/2 = \nu_{02}/2.$$

In the frame rotating at ν_d and under the RWA, the control Hamiltonian is

$$H = \frac{2\pi\nu_{\text{Ref}}}{2} (u(t) a^\dagger + u^*(t) a) - \frac{2\pi\alpha}{4} a^\dagger a + \frac{2\pi\alpha}{2} a^{\dagger 2} a^2.$$

To capture leakage, we simulate the system with four energy levels, with a and $a^\dagger$ truncated accordingly. Starting from $|\psi(0)\rangle = |0\rangle$, the objective is to optimize $u(t)$ to transfer the state to $|2\rangle$, such that $|\langle 2|\psi(T)\rangle|^2 = 1$. The time is discretized into 200 steps of 0.1 ns, corresponding to $T = 20$ ns. As in Example 1, this problem is underconstrained, and we again minimize the pulse energy to select a minimum-amplitude solution.

Problem setup.

```
system = Transmon(
    n_levels=4,
    anharmonicity=-200e6,
    frequency=5e9,
    carrier_frequency=5e9 - 200e6/2, # Two-photon resonance
)
target = StateTarget(initial=[1, 0, 0, 0], target=[0, 0, 1, 0])
problem = OptimizationProblem(system, target, n_steps=200, dt=0.1e-9,
                               reference_amplitude=100e6)
```

Note that the computational subspace does not need to be specified for a `StateTarget`: the initial and target states are defined in the full Hilbert space.

Initial pulse guess. We use a symmetric/antisymmetric random low-order Fourier series:

```
pulse0 = initial_guess.random_symmetric_antisymmetric(problem)
```

Optimization. We again search for a minimum-energy solution:

```
optimizer = GrapeOptimizer(max_iter=1000, minimize_energy=True)
result = optimizer.optimize(problem, pulse0)
```

Results. The pulse is visualized with `result.plot_pulse(compare_to_initial=False)`. The resulting pulse is shown in Fig. 2, together with the transition probabilities $P_{0\rightarrow 1}$, $P_{0\rightarrow 2}$ (target) and $P_{0\rightarrow 3}$ (leakage), obtained from `result.population(0, k)`. The complete script is provided in `examples/02_two_photon_transition.ipynb`.

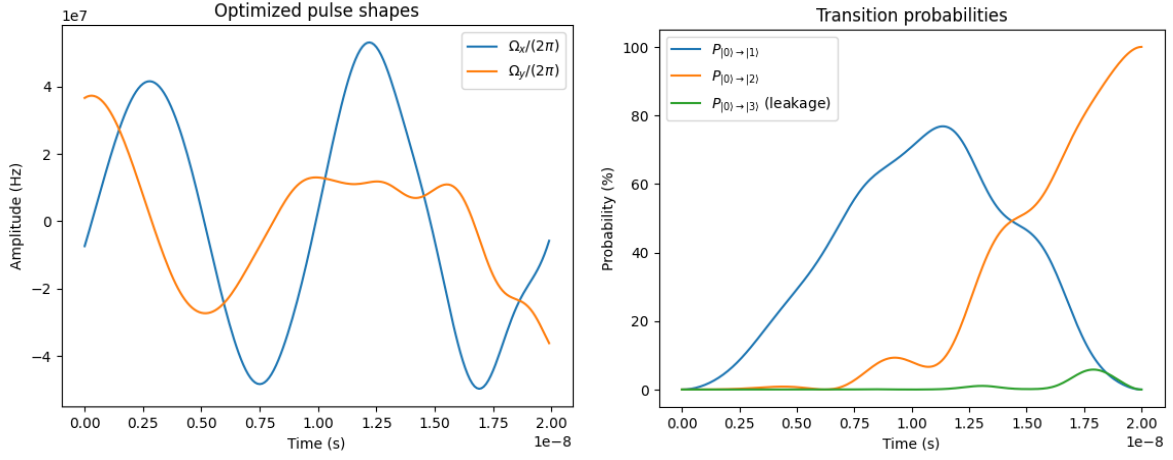


Figure 2: Left: Pulse amplitudes (in Hz) implementing a two-photon transition from $|0\rangle$ to $|2\rangle$. Right: Transition probabilities as a function of time.

Example 3: Analytically Shaped Hadamard Gate

`examples/03_analytic_pulse_shaping.ipynb`

General problem. We consider the same three-level, on-resonance system as in Example 1, now with $\alpha = -200$ MHz, and target a Hadamard gate,

$$U_T = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix},$$

on the computational subspace spanned by $|0\rangle$ and $|1\rangle$. In contrast to Examples 1 and 2, the pulse shape is restricted to a basis of analytical functions:

$$u_x(t) = \sum_{k=1}^3 a_{x_k} \sin\left(\frac{(2k-1)\pi t}{T}\right),$$

$$u_y(t) = \sum_{k=1}^3 a_{y_k} \cos\left(\frac{(2k-1)\pi t}{T}\right),$$

and the optimizer searches for the coefficients $\{a_{x_k}, a_{y_k}\}$ that realize the target gate. The control pulse is discretized into 160 time steps of 0.1 ns, i.e. a total duration of 16 ns.

Problem setup. Instead of optimizing the pulse directly at each time step, the control fields are described as a combination of analytical functions, passed to `OptimizationProblem` as `ux_basis` and `uy_basis`:

```
n_steps, n_coeffs = 160, 3
tau = np.linspace(0, 1, n_steps)
fx = np.array([np.sin((2*n+1)*np.pi*tau) for n in range(n_coeffs)
])
fy = np.array([np.cos((2*n+1)*np.pi*tau) for n in range(n_coeffs)
])

system = Transmon(n_levels=3, anharmonicity=-200e6)
target = GateTarget(matrix=np.array([[1, 1], [1, -1]])/np.sqrt(2),
    subspace=[0, 1])
```

```
problem = OptimizationProblem(system, target, n_steps=n_steps, dt
                             =0.1e-9,
                             ux_basis=fx, uy_basis=fy)
```

With this setup, the optimizer no longer works on u_x and u_y at each time step, but only on the coefficients a_{x_k} and a_{y_k} .

Initial pulse guess. We use a random low-order Fourier guess for the coefficients:

```
pulse0 = initial_guess.random_fourier(problem, n_coeffs=n_coeffs)
```

Optimization.

```
result = GrapeOptimizer(max_iter=500).optimize(problem, pulse0)
```

Results. The pulse component $u_x(t)$ and $u_y(t)$ are recovered from the coefficients via the basis stored on `problem` and visualized with `result.plot_pulse()`. The resulting pulse is shown in Fig. 3, together with the transition probabilities (including leakage to $|2\rangle$). The complete script is provided in `examples/03_analytic_pulse_shaping.ipynb`.

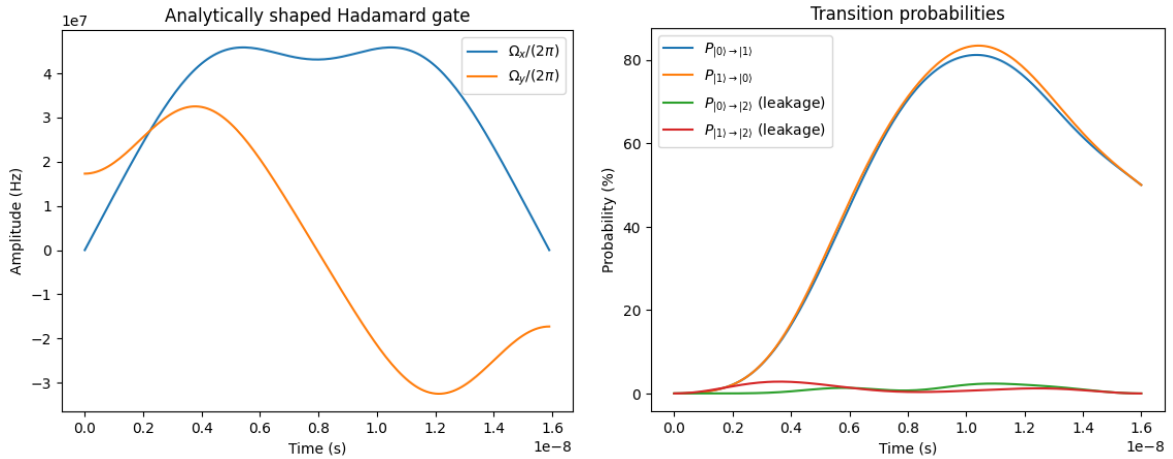


Figure 3: Left: Analytically shaped pulse implementing a Hadamard gate. Right: Transition probabilities, including leakage to $|2\rangle$.

Example 4: Robust Fourier Pulse

examples/04_robust_fourier_pulse.ipynb

General problem. We consider a transmon system subject to both pulse amplitude deviations and detuning errors. Within the rotating-wave approximation, the system Hamiltonian is

$$H(\epsilon, \delta, t) = \frac{2\pi\nu_{\text{Ref}}}{2} (1 + \epsilon) \left(u(t) a^\dagger + u^*(t) a \right) + 2\pi(\nu_q - \nu_d + \delta) a^\dagger a + \frac{2\pi\alpha}{2} a^{\dagger 2} a^2,$$

where $\alpha = -200$ MHz, $\nu_{\text{Ref}} = 5$ MHz, and $\nu_q = \nu_d = 5$ GHz. Here ϵ (dimensionless) models a relative drive-amplitude error, while δ (in Hz) is a detuning error. The goal is to design a control pulse $u(t)$ implementing an $X_{\pi/2}$ gate,

$$U_T = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -i \\ -i & 1 \end{pmatrix},$$

on the computational subspace $\{|0\rangle, |1\rangle\}$, that remains accurate for all (ϵ, δ) within $\epsilon \in [-0.1, 0.1]$, $\delta \in [-0.2, 0.2]$ MHz. In practice, ϵ and δ are discretized on a 5×5 grid, and the optimizer minimizes the average infidelity over the grid,

$$J = \frac{1}{N_\epsilon N_\delta} \sum_{n=1}^{N_\epsilon} \sum_{m=1}^{N_\delta} \left(1 - \frac{1}{2} \left| \text{Tr} \left(U_T^\dagger U_{nm}^{01}(T) \right) \right|^2 \right).$$

As in Example 3, the pulse is restricted to a Fourier basis,

$$u_x(t) = \sum_{k=1}^5 a_k \sin\left(\frac{(2k-1)\pi t}{T}\right), \quad u_y(t) = \sum_{k=1}^5 b_k \sin\left(\frac{2k\pi t}{T}\right),$$

with an amplitude constraint $\nu_{\text{Ref}} u_x(t), \nu_{\text{Ref}} u_y(t) \leq 15$ MHz. The control pulse is discretized into 200 steps of 0.5 ns (100 ns total).

Problem setup. The robustness grid is built with `Transmon.sweep`, which meshes the swept keyword arguments and remembers the grid shape for later fidelity-map plotting:

```
Na, Nd = 5, 5
amp_error = np.linspace(-0.1, 0.1, Na)      # Amplitude error
[-10%, 10%]
detuning = 0.2e6 * np.linspace(-1, 1, Nd)   # Detuning [-0.2 MHz,
0.2 MHz]

system = Transmon.sweep(
    frequency=5e9 + detuning,
    amplitude_scale=1 + amp_error,
    anharmonicity=-200e6,
)
target = GateTarget(matrix=np.array([[1, -1j], [-1j, 1]])/np.sqrt
(2), subspace=[0, 1])

n_steps, n_coeffs = 200, 5
fx = symmetric_fourier_basis(n_steps, n_coeffs)
fy = antisymmetric_fourier_basis(n_steps, n_coeffs)
```

```

problem = OptimizationProblem(system, target, n_steps=n_steps, dt
                             =0.5e-9,
                             reference_amplitude=5e6, ux_basis=
                             fx, uy_basis=fy)

```

Initial pulse guess.

```

pulse0 = initial_guess.cos_drag(problem, theta=np.pi/2)

```

Optimization. We impose the amplitude constraint via `max_in_phase/max_quadrature` :

```

optimizer = GrapeOptimizer(max_iter=2000, max_in_phase=15e6,
                           max_quadrature=15e6)
result = optimizer.optimize(problem, pulse0)

```

Results. The pulse is shown in the left panel of Fig. 4. To visualize robustness, the same optimized pulse is re-evaluated on a finer (δ, ϵ) grid via `problem.with_system(fine_system)` and `analysis.fidelity_map(...)`, producing the fidelity map on the right. The complete script is provided in `examples/04_robust_fourier_pulse.ipynb`.

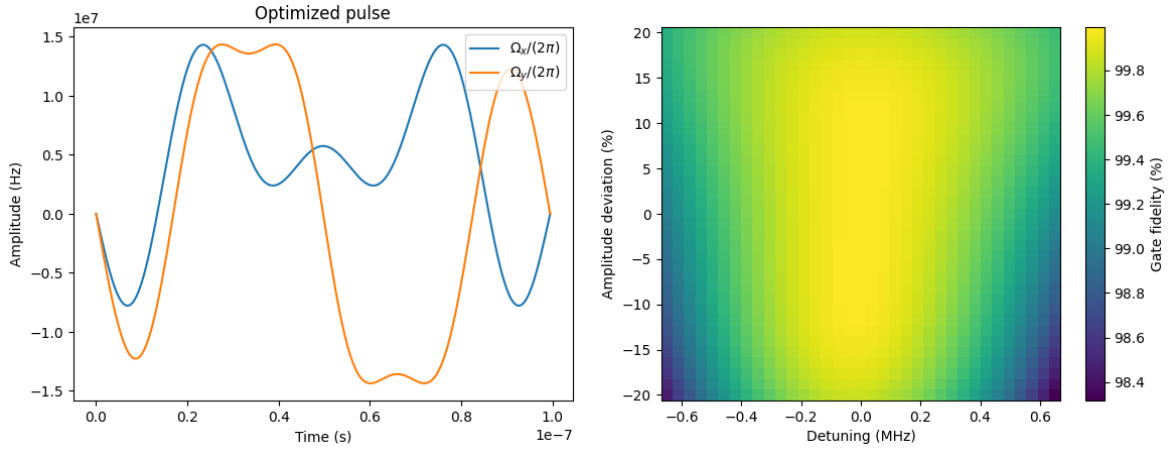


Figure 4: Left: Pulse amplitudes (in Hz) implementing a robust $X_{\pi/2}$ gate. Right: Gate fidelity as a function of detuning and amplitude deviation, evaluated on a finer grid than the one used for optimization.

Example 5: Selective Pulse

examples/05_selective_pulse.ipynb

General problem. We consider a two-level quantum system with dynamics governed by

$$H(\delta, t) = \frac{2\pi\nu_{\text{Ref}}}{2} (u(t) a^\dagger + u^*(t) a) + 2\pi(\nu_q - \nu_d + \delta) a^\dagger a,$$

where $\nu_q = \nu_d = 5$ GHz and the detuning δ takes one of two values, ± 2.5 MHz. Since only the two lowest levels are considered, the ladder operators are truncated as $a = |0\rangle\langle 1|$, $a^\dagger = |1\rangle\langle 0|$. The goal is to design a control pulse $u(t)$ that maps $|0\rangle$ to a *different* outcome depending on the detuning:

$$\begin{cases} |0\rangle \rightarrow |0\rangle & \text{if } \delta = -2.5 \text{ MHz}, \\ |0\rangle \rightarrow |1\rangle & \text{if } \delta = +2.5 \text{ MHz}, \end{cases}$$

subject to the amplitude constraint $|u(t)|\nu_{\text{Ref}} \leq 8$ MHz. The control pulse is discretized into 120 time steps of 1 ns (120 ns total). Selectivity is achieved by simultaneously optimizing over the two detuning values as two *ensemble members* of the same **Transmon**, each carrying its own target state.

Problem setup. Both ensemble members start in $|0\rangle$; member 0 (detuning -2.5 MHz) targets $|0\rangle$, member 1 ($+2.5$ MHz) targets $|1\rangle$:

```
system = Transmon(
    n_levels=2,
    frequency=5e9 + np.array([-2.5e6, 2.5e6]),
    carrier_frequency=5e9,
)
target = StateTarget(initial=[[1, 0], [1, 0]], target=[[1, 0], [0, 1]])
problem = OptimizationProblem(system, target, n_steps=120, dt=1e-9)
```

Initial pulse guess. We start from a constant drive along x :

```
pulse0 = initial_guess.zero(problem)
pulse0.ux[:] = 1.0
```

Optimization.

```
optimizer = GrapeOptimizer(max_iter=2000, max_amplitude=8e6)
result = optimizer.optimize(problem, pulse0)
```

Results. The resulting pulse resembles a Ramsey experiment, with the pulse switched off during an intermediate free-evolution window (Fig. 5, left), during which the two detunings accumulate opposite phases that are resolved by the final pulse segment into the two different target outcomes (right panel). This solution is analogous to the time-optimal solution reported for the equivalent (swapped-target) problem in Phys. Rev. A 98, 043421. The complete script is provided in `examples/05_selective_pulse.ipynb`.

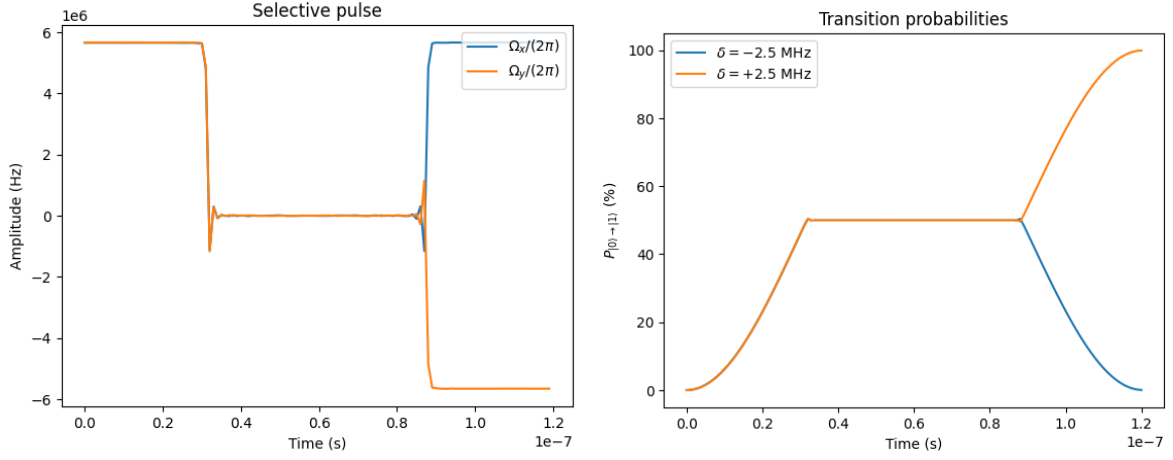


Figure 5: Left panel: Optimal pulse shape in Hz. Right panel: Transition probability $P_{0 \rightarrow 1}$ as a function of time, for both detunings.

Example 6: Time-Dependent Detuning Robustness

examples/06_time_dependent_detuning.ipynb

General problem. We revisit the leakage-free NOT gate of Example 1 ($\alpha = -100$ MHz, three levels), but now require robustness against a time-dependent detuning of the form

$$\delta(t) = A \cos(2\pi\omega t),$$

over a grid of $\omega \in [0, 10]$ MHz and $A \in [-5, 5]$ MHz. Because `Transmon.frequency` accepts a 2-D array of shape `(n_ensemble, n_steps)`, each grid point (ω_n, A_m) becomes one ensemble member with its own detuning *trace* $\delta_{nm}(t)$, rather than a single constant offset as in Example 4. As in Example 4, the optimizer minimizes the ensemble-averaged infidelity over the grid. The pulse is restricted to a symmetric/antisymmetric Fourier basis with 10 harmonics per quadrature and constrained to $(u_x^2(t) + u_y^2(t))^{1/2} \nu_{\text{Ref}} \leq 20$ MHz. The control pulse is discretized into 150 steps of 1 ns (150 ns total).

Problem setup.

```
n_steps, dt = 150, 1e-9
tc = np.arange(n_steps) * dt
Nw, NA = 5, 5
w = np.linspace(0, 10e6, Nw)
A = np.linspace(-5e6, 5e6, NA)
wGrid, AGrid = np.meshgrid(w, A)
detuning = AGrid.ravel()[:, None] * np.cos(2*np.pi*wGrid.ravel()
   [:, None] * tc[None, :])

system = Transmon(n_levels=3, frequency=5e9 + detuning,
    carrier_frequency=5e9, anharmonicity=-100e6)
system.grid_shape = wGrid.shape # Remember the grid for
    later plotting
system.grid_axes = {"w": w, "A": A}

target = GateTarget(matrix=[[0, 1], [1, 0]], subspace=[0, 1])
```

```

n_coeffs = 10
fx = symmetric_fourier_basis(n_steps, n_coeffs)
fy = antisymmetric_fourier_basis(n_steps, n_coeffs)
problem = OptimizationProblem(system, target, n_steps=n_steps, dt=
    dt,
                                ux_basis=fx, uy_basis=fy)

```

Initial pulse guess.

```

pulse0 = initial_guess.cos_drag(problem, theta=np.pi)

```

Optimization.

```

optimizer = GrapeOptimizer(max_iter=2000, max_amplitude=20e6)
result = optimizer.optimize(problem, pulse0)

```

Results. The optimized pulse is shown in Fig. 6. As in Example 4, the optimized pulse and the initial DRAG guess are then both re-evaluated on a finer 55×55 (ω, A) grid, via `problem.with_system(fine_system)` and `analysis.fidelity_map(...)`, to produce the fidelity maps in Fig. 7: the optimized pulse maintains high fidelity over a much wider region of the (ω, A) plane than the DRAG guess. The complete script is provided in `examples/06_time_dependent_detuning.ipynb`.

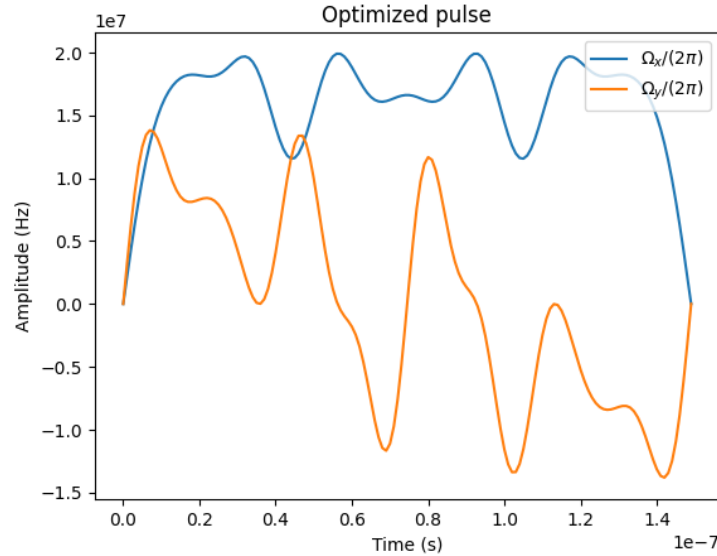


Figure 6: Optimized pulse, robust to a time-dependent detuning $\delta(t) = A \cos(2\pi\omega t)$.

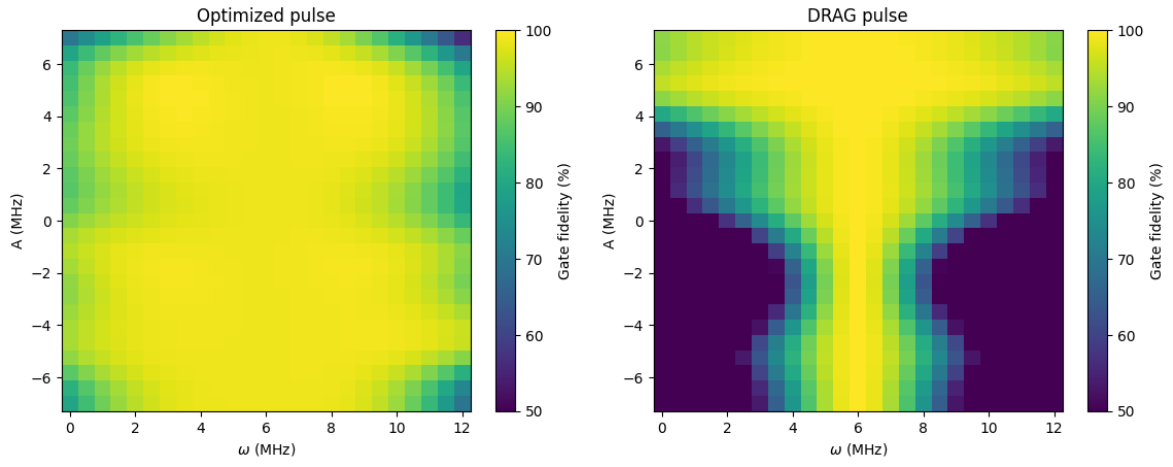


Figure 7: Gate fidelity as a function of (ω, A) , evaluated on a finer grid than the one used for optimization. Left: optimized pulse. Right: initial DRAG guess.